

```

1  (*
2                                     CS51 Lab 7
3                                     Modules and Abstract Data Types
4  *)
5
6  (* Objective: This lab practices concepts of modules, including files
7  as modules, signatures, and polymorphic abstract data types.
8
9  There are three total parts to this lab. Please refer to the following
10 files to complete all exercises:
11
12     lab7_part1.ml -- Part 1: Implementing modules
13 -> lab7_part2.ml -- Part 2: Files as modules (this file)
14     lab7_part3.ml -- Part 3: Polymorphic abstract types and
15                           Interfaces as abstraction barriers
16  *)
17
18  (*=====
19  Part 2: Files as modules
20
21  A useful feature of OCaml is that it *automatically* wraps all of the
22  functions and values that are defined in a single file into a module
23  named after that file. The module name is the name of the file with
24  the first letter capitalized. This functionality is in addition to the
25  manual definition of modules as you've just used in Part 1, and it is
26  a convenient way of separating code into separate namespaces when
27  writing a large program.
28
29  There are other source files included in this lab, other than the
30  `lab7_partn.ml` files. The file `color.ml` contains an implementation
31  of a system for managing colors. (Recall from lab 5 the idea of colors
32  as consisting of values for three color channels -- red, green, and
33  blue.) Take a look at `color.ml` to see what functions and values it
34  contains, including a type for colors, and a means for converting
35  values for the three color channels into the abstract `color` type,
36  extracting the channels individually from colors, and converting some
37  standard color names to this color representation.
38
39  With the exception of Exercise 2A, you will need to modify *only*
40  the file `color.ml` to complete the exercises below.
41
42  A digression on accessing other modules:
43
44  You'll want to test this part of the lab using `ocamlbuild`, for
45  instance, with
46
47      % ocamlbuild -use-ocamlfind lab7_part2.byte
48      % ./lab7_part2.byte
49
50  The `ocamlbuild` command should automatically find modules that
51  you've written that reside in the same directory as your source,
52  compile those additional files, and link them to your compiled

```

program. You can then access functions from those files under the module name, which (again) is the name of the file with the first letter capitalized. For instance, if ``color.ml`` is in the same directory as ``lab7_part2.ml`` (which it probably is), ``ocamlbuild`` will find it and use it, since it is referenced through expressions like ``Color.red`` and the like.

On the other hand, if you're testing with a top-level REPL, like `utop` or `ocaml`, it will *not* automatically find those modules and evaluate them for you. However, you can inform the REPL about those modules manually yourself with the ``#mod_use`` directive, like this:

```
# #mod_use "color.ml" ;;
```

allowing you to then refer to elements of the module as, for instance,

```
# Color.color_named ;;
- : Color.color_name -> int = <fun>
```

(Note the capitalized module name, as discussed above.)

Keep in mind however that the ``#mod_use`` and ``#use`` directives look only at their argument files, not at any corresponding ``mli`` files, so the modules being used will *not* be restricted to the signatures in the corresponding ``mli`` files. For that, you'll have to use the compiler (with ``ocamlbuild`` for instance).

.....
Exercise 2A: Replace the ``0`` in the expression below with an expression that extracts the red channel of the color named ``Red``, naming the result ``red_channel_value``. (The expression will be constructed using values in the ``Color`` module.)
.....*)

```
let red_channel_value : int = 0 ;;
```

(* Let's investigate one way that a signature can be useful. Although ``color.ml`` contains an implementation of a basic color module, the implementation is unintuitive and obscure -- truly **horrid** in fact. (We did that on purpose.) You will want to change the implementation of ``color.ml``, rewriting it wholesale. At the same time, you'll want to guarantee to users of the module (like this file itself!) that the functionality from the "outside" point of view stays the same; the way to do this is through module **signatures**.

.....
Exercise 2B: Add a file ``color.mli``, in which you define an appropriate signature for the ``Color`` module. Consider which types and values you want revealed to the user and which you would prefer to be hidden.

HINT: You'll want users of the module to know that there is a ``color`` type, but not how it's implemented. You'll want users of

```

107     the module to know that there is a `color_name` type, and also
108     what its enumerated constructors are, so that users can invoke
109     those colors.
110
111 Once you have `color.mli` implemented, you should still be able to
112 compile `color.ml` and run `color.byte`.
113 .....*)
114
115
116 (*.....
117 Exercise 2C:
118
119 In the file `color.ml`, modify the implementation of a color module as
120 you see fit. Make the design choices that you think would be best for a
121 color module implementation. In particular, you should be able to come
122 up with a solution that is *much* SIMPLER, clearer, and more transparent
123 than the one currently in `color.ml`. (Frankly, I recommend throwing out
124 the `color.ml` contents entirely and starting over.)
125
126 To pass the unit tests, you'll want the RGB values for the colors in the
127 `color_name` type to have the following values from lab 5:
128
129     R | G | B | Color
130     ---|---|---|-----
131     255 | 0 | 0 | Red
132     0 | 255 | 0 | Green
133     0 | 0 | 255 | Blue
134     255 | 165 | 0 | Orange
135     255 | 255 | 0 | Yellow
136     75 | 0 | 130 | Indigo
137     240 | 130 | 240 | Violet
138
139 .....*)
140
141
142 (* Here's the payoff: A user who uses the `color` module, by virtue of
143 having to stay within the `color.mli` interface, will not notice *any
144 difference at all* between the two implementations of `color.ml`, the
145 horrid one we provided and the elegant one you've developed. The
146 underlying implementation can be changed any time in any way, so long
147 as the functionality provided stays consistent with the signature.
148
149 Correspondingly, consider the `List` module that you are familiar with
150 by now. You never needed to worry about how the `List` module was
151 implemented in order to use it; you only needed to understand the
152 interface.
153
154 Compartmentalization ftw! *)

```